

Informe de Proyecto: Packet Flow

Algoritmos y Estructuras de Datos - UCU

Integrantes: Emanuel Shu, Ignacio Rodriguez, Nicolas Lozano

1. Introducción

PacketFlow es un sistema de simulación de red por consola desarrollado en Java. Su objetivo principal es modelar la fragmentación de datos, transmisión y posterior reconstrucción utilizando estructuras de datos lineales (como listas y colas).

El proyecto fue diseñado como parte de un entorno educativo para demostrar cómo los mensajes grandes se dividen en paquetes más pequeños, se envían a través de una red con capacidad limitada, y finalmente se reconstruyen en el lado del receptor, abordando el desafío de ordenar paquetes que pueden llegar desordenados.

2. Arquitectura del Sistema

El sistema está compuesto por varias clases fundamentales ubicadas en el paquete `com.aed.packetflow.model` y una clase principal `Main` para la interacción con el usuario mediante consola.

- **Message:** Representa un mensaje lógico que se desea enviar. Mantiene el estado (`MessageStatus`: INCOMPLETO, COMPLETO, RECONSTRUIDO), la prioridad, y valida cuándo se han recibido todos sus paquetes.
- **Packet:** Representa un fragmento de un mensaje. Transporta la información de su orden secuencial y tamaño para permitir su correcta reconstrucción.
- **Network:** Simula el medio de transmisión. Recibe mensajes, los fragmenta según el `maxPacketSize`, los encola, y maneja el descarte si la capacidad es superada.
- **Reconstructor:** Componente encargado de reordenar los paquetes de un mensaje completo basándose en su número de secuencia.
- **Main:** Provee el menú interactivo que permite configurar la red, enviar mensajes, consultar el estado detallado de la transmisión y eliminar mensajes en tránsito.

3. Estructuras de Datos Utilizadas

El proyecto aprovecha las interfaces de colecciones estándar de Java, restringiéndose a estructuras lineales como especifica la letra del proyecto:

- **Queue<Packet> (LinkedList):** Utilizada en `Network` para modelar la cola de tránsito de los paquetes. Representa fielmente el comportamiento de "primero en entrar, primero en salir" (FIFO) típico de las transmisiones de red simples.
- **List<Message> / List<Packet> (LinkedList):** Utilizadas para almacenar el registro general de mensajes en la red y los paquetes recibidos dentro de cada mensaje. Proveen la flexibilidad necesaria para buscar elementos por ID y añadir fragmentos dinámicamente.

4. Flujo de Ejecución y Funcionalidades

1. **Creación y Fragmentación:** El usuario solicita el envío de un mensaje. La red evalúa el tamaño total y crea N paquetes necesarios, asignándoles un número de secuencia.
2. **Transmisión y Descarte:** Los paquetes son encolados. Si se supera la capacidad máxima de la red, los paquetes excedentes se descartan, simulando pérdida de datos.
3. **Interrupción (Eliminación):** El usuario puede buscar un mensaje por su ID y eliminarlo. Esto elimina el registro lógico y purga todos sus paquetes huérfanos que aún se encontraban en la cola de tránsito.
4. **Recepción y Reensamblado:** Los paquetes se extraen de la red y se asignan a su mensaje original. Una vez que todos llegan, el mensaje pasa a estado COMPLETO. Finalmente, el `Reconstructor` ordena los paquetes por secuencia y lo marca como RECONSTRUIDO.

5. Decisiones de Diseño, Asunciones y Dificultades

Durante el desarrollo, se tomaron varias decisiones arquitectónicas y se asumieron ciertas condiciones para acotar el alcance del simulador:

- **Asunciones:** Se asumió que los identificadores (ID) ingresados por el usuario para los mensajes son únicos. Respecto a los requerimientos opcionales de la letra, se agregó el soporte para "prioridad" en la clase Message, pero se decidió mantener la transmisión puramente FIFO para simplificar la lógica base. No se implementó la carga automática desde archivo CSV.
- **Dificultades y Soluciones:** Una dificultad técnica importante fue la implementación de la eliminación de un mensaje (Requerimiento 1.c). Como la cola de tránsito (`Queue`) no permite eliminar elementos arbitrarios fácilmente sin romper su estructura, se optó por una solución donde, al eliminar un mensaje, se extraen todos los paquetes válidos a una nueva cola temporal, descartando los del mensaje eliminado, y luego se reemplaza la cola original.

- **Reordenamiento:** Para simular que los paquetes llegan desordenados y reensamblarlos (Requerimiento 3.a), se delegó la responsabilidad a la clase `Reconstructor`, la cual utiliza un `Comparator` basado en el atributo `sequenceNumber` de cada paquete.

6. Validación y Pruebas (JUnit)

Para asegurar la robustez de la lógica de red y el manejo de estructuras de datos (Requerimiento 4), se implementó una suite de pruebas unitarias en la clase `PacketFlowTest`:

- **Fragmentación:** (`testSendMessageFragmentation`) Verifica que la matemática de división de paquetes genere la cantidad exacta esperada según el tamaño máximo.
- **Límite de Capacidad:** (`testNetworkCapacityLimit`) Confirma que la red descarta correctamente los paquetes sobrantes cuando se satura su capacidad máxima.
- **Reconstrucción Desordenada:** (`testReceiveAndReconstruct`) Simula la llegada de paquetes en un orden inverso al enviado, validando que el `Reconstructor` logre ordenarlos correctamente y cambiar el estado final del mensaje.
- **Eliminación de Mensajes:** (`testDeleteMessage`) Comprueba que al eliminar un mensaje, sus paquetes asociados desaparecen efectivamente del tránsito de la red.

7. Conclusión

`PacketFlow` demuestra los principios fundamentales de redes informáticas, específicamente la relación entre la capa de red (fragmentación/transmisión) y la capa de transporte (reensamblaje/estado). Mediante el uso de programación orientada a objetos, pruebas unitarias y estructuras de datos lineales simples pero efectivas en Java, se logró resolver un escenario complejo de sincronización y validación de datos.